

Generative AI & RAG Study Guide

Internship Task 3 — Retrieval-Augmented Generation

This document is designed as a realistic technical knowledge base for a RAG project. It contains document-specific concepts and a fictional benchmark example that can be used to demonstrate the difference between a plain LLM and a retrieval-grounded system.

1. Introduction to Generative AI

Generative AI refers to artificial intelligence systems that can create new content such as text, code, images, audio, and structured data. Large language models are a major class of generative AI systems. They generate responses by predicting useful sequences of tokens from an input context.

An important limitation of a language model is that its answer is not automatically connected to a private company knowledge base. If a user asks about information contained only in a private document, the model may not know the answer or may produce an unsupported answer.

Retrieval-Augmented Generation, commonly called RAG, addresses this limitation by retrieving relevant information from an external knowledge source before generation. The retrieved information is placed into the model's context so that the answer can be grounded in the supplied material.

2. What is RAG?

RAG is an architecture that combines information retrieval with language generation. Instead of asking the language model to answer using only its internal learned knowledge, the application first searches a document collection for relevant passages.

A typical RAG pipeline has five major stages: document ingestion, chunking, embedding, vector storage and retrieval, followed by prompt construction and generation. Each stage solves a different problem.

Document ingestion converts source files into machine-readable text. Chunking divides long text into manageable pieces. Embeddings represent chunks as numerical vectors. A vector database stores those representations. Retrieval finds chunks whose semantic meaning is close to the user's question. Generation then uses those retrieved chunks as context.

3. Document Ingestion

Document ingestion is the process of bringing source material into a RAG application. Common sources include PDF files, Markdown files, HTML pages, text files, manuals, reports, and internal knowledge documents.

A good ingestion pipeline should preserve useful headings, paragraphs, lists, and page boundaries when possible. Poor extraction can damage the later retrieval process because embeddings are generated from the extracted text.

For this project, a PDF-based collection is suitable because PDFs provide enough realistic content to demonstrate a complete ingestion workflow. The application should read the PDF, extract its text, and pass the resulting documents to the chunking stage.

4. Chunking Strategy

Chunking means dividing a large document into smaller passages. A single huge document is difficult to retrieve precisely because a query may match only a small portion of it. Extremely tiny chunks can also lose the surrounding context needed to understand an idea.

A practical starting point is approximately 300 to 800 tokens per chunk, with overlap between neighboring chunks. Overlap helps preserve context when an important sentence occurs near a chunk boundary.

Chunk size should be treated as an engineering parameter rather than a universal constant. Technical documentation with short sections may benefit from smaller chunks, while explanatory prose may work well with larger chunks.

Before using the chunks for retrieval, inspect several examples manually. A useful chunk normally contains a coherent topic rather than an arbitrary fragment. The project should record the number of chunks created so that the ingestion process is easy to verify.

5. Embeddings

An embedding is a numerical representation of text. Texts with related meaning tend to have vectors that are close according to an appropriate similarity measure. This allows a retrieval system to compare a user's question with document chunks based on meaning rather than only exact keyword matching.

For example, a question about reducing hallucinations can retrieve a passage discussing grounding, citations, and evidence even if the exact phrase used in the passage differs from the question.

The same embedding model or a compatible embedding space should be used for document chunks and user queries. The resulting vectors are stored with metadata such as source file, page number, and chunk identifier when that information is available.

6. Chroma Vector Database

Chroma is a vector database that can be run locally, making it convenient for a learning project. It stores embeddings and associated document information and supports similarity search.

A local Chroma collection can contain the text of each chunk, its embedding, and metadata. When the application receives a question, the question is embedded and compared against the stored vectors.

Chroma is especially useful for this internship task because it avoids the account and cloud configuration required by some managed vector databases. A persistent local directory can be used so that the vector store does not need to be rebuilt for every application run.

7. Retrieval

Retrieval is the stage where the system searches the vector database for the most relevant chunks. The user's question is converted into an embedding and compared with document embeddings.

Top-k retrieval means returning the k highest-ranked chunks. A value such as 3 or 5 is often a useful starting point, but the best value depends on document size and query complexity.

Retrieval should be tested independently before connecting it to the language model. Printing the retrieved chunk text and metadata makes it possible to confirm that the system is finding relevant evidence instead of random content.

8. Grounded Generation

After retrieval, the application constructs a prompt containing the retrieved context and the user's question. The model is instructed to answer using the supplied context.

A useful grounded prompt tells the model to rely on the provided material, avoid inventing unsupported facts, and state when the context does not contain enough information. This is important because retrieval alone does not guarantee that the final answer will be faithful.

The generated response should be evaluated against the source passages. The goal is not merely to produce a fluent answer, but to produce an answer supported by the retrieved evidence.

9. RAG vs Plain LLM

A plain LLM call sends the question directly to the model without retrieving the project's documents. Such a call can answer general questions using learned knowledge, but it cannot reliably know arbitrary private facts introduced only in the project's document set.

A strong demonstration uses a document-specific question whose answer appears only in the project documents. For example, the document can define a fictional internal metric, named framework, or project rule. The plain model should not be expected to know this invented fact.

The RAG version retrieves the passage containing the unique fact and supplies it to the model. The README can show the question, the plain-model response, the retrieved evidence, and the final RAG response.

10. Prompt Design for RAG

Prompt design affects how effectively retrieved information is used. The prompt should clearly separate instructions, retrieved context, and the user question.

One useful structure is: system instructions first, context second, and the question last. Context can be labeled with source identifiers so that the model can distinguish evidence from instructions.

The prompt should discourage unsupported completion. If the answer is not present in the retrieved context, the model should say that the supplied documents do not contain enough information rather than confidently inventing an answer.

11. Metadata and Sources

Metadata improves traceability. Useful fields include source filename, page number, section heading, chunk number, and document identifier.

When a response is generated, metadata can be used to show which document passages were retrieved. This makes debugging easier and provides users with a way to understand where an answer came from.

For an internship demonstration, printing source metadata alongside retrieved chunks is valuable evidence that the retrieval layer is functioning correctly.

12. Evaluation of Retrieval

Retrieval quality can be evaluated with a small set of test questions. Each question should have an expected relevant section. Run the questions and inspect the top retrieved chunks.

Useful checks include relevance, coverage, and noise. Relevance asks whether the returned passages discuss the question. Coverage asks whether the necessary evidence is present. Noise asks whether too many unrelated passages are returned.

Evaluation should be repeated after changing chunk size, overlap, or top-k. This turns RAG development into an iterative engineering process rather than a one-time coding exercise.

13. Common RAG Failure Modes

One common failure occurs when chunking removes necessary context. Another occurs when the retrieval query does not match the terminology used in the document. A third occurs when the correct chunk is retrieved but the prompt fails to tell the model how to use it.

Other problems include duplicate chunks, poor PDF text extraction, excessive context, irrelevant top-k results, and answers that contain information not supported by the documents.

Debugging should proceed in stages: inspect extracted text, inspect chunks, inspect retrieved results, inspect the final prompt, and finally inspect the generated response. Testing the complete pipeline without checking intermediate stages makes errors harder to locate.

14. Security and Privacy

RAG systems may process confidential documents. Access controls, safe storage, logging policies, and API-key management are therefore important in real applications.

API keys should never be hard-coded into source files or committed to public repositories. Environment variables or the deployment platform's secret-management facility should be used.

A public internship repository should contain sample or non-sensitive documents only. Private company information, credentials, personal records, and confidential material should not be uploaded to a public GitHub repository.

15. Project Architecture

A simple architecture for this internship project is: PDF files -> loader -> text splitter -> embeddings -> Chroma collection -> question embedding -> similarity search -> retrieved context -> Gemini prompt -> answer.

The ingestion process can be kept separate from the query process. A build script can create the vector store once, while the application loads the existing store when answering questions.

Separating ingestion from querying makes the application faster at runtime and makes the code easier to explain during an interview.

16. Suggested Python Project Structure

A clean project can contain files such as ingest.py, rag.py, app.py, requirements.txt, README.md, .env.example, data/, and chroma_db/. The data directory contains the source document and the Chroma directory contains the local vector store.

ingest.py is responsible for loading, splitting, embedding, and storing documents. rag.py can contain retrieval and prompt-generation functions. app.py can provide a simple command-line or Streamlit interface.

The .gitignore file should exclude .env and other secret or generated files as appropriate. The README should explain setup, ingestion, retrieval, testing, and the RAG comparison.

17. Testing Questions

Test questions should include direct factual questions, paraphrased questions, questions requiring information from multiple chunks, and an out-of-scope question.

A document-specific question is particularly important for the internship requirement. Create or select a fact that exists only inside the supplied document set. Then ask the same question first without retrieval and again with retrieval.

Record the retrieved chunks and compare the two answers. The demonstration should make it obvious that the RAG system is using the external document rather than relying only on the model's general knowledge.

18. Practical Example of Grounding

Suppose the document contains a fictional section stating that the Aurora RAG benchmark uses a threshold of 0.83 and a retrieval depth of 4. These values are project-specific and are not general facts.

A plain language-model call may say it does not know the project's benchmark settings or may guess. The RAG pipeline can retrieve the section containing the settings and answer with the exact values from the document.

This kind of test is useful because it isolates the contribution of retrieval. The document-specific fact acts as evidence that the external knowledge source was actually used.

19. Improving RAG Quality

RAG quality can be improved through better document parsing, semantic chunking, appropriate overlap, stronger embeddings, metadata filtering, reranking, query rewriting, and careful prompt design.

Another useful technique is hybrid retrieval, which combines semantic similarity with keyword-based matching. This can help when a question contains an exact product name, code, identifier, or specialized term.

For a beginner internship project, however, a simple and understandable pipeline is preferable to adding advanced components without understanding them. A strong basic implementation is easier to test and defend.

20. Final Project Checklist

The completed project should contain a real document collection of at least 10 to 20 pages, sensible chunks with overlap, embeddings, a Chroma or Pinecone vector database, a tested retrieval function, and a generation step that uses retrieved context.

The repository should include evidence in the README. Show a sample question, the retrieved chunks, the plain LLM answer, and the RAG answer. Clearly explain why the document-specific question demonstrates the value of retrieval.

Before submission, verify that the GitHub repository is public, that secrets are not committed, that the README contains setup instructions, and that another person could understand the architecture by reading the repository.